Tema 7. Triggers, procedimientos y funciones en MySQL

JOSÉ JUAN SÁNCHEZ HERNÁNDEZ

IES Celia Viñas (Almería) - 2017/2018

Índice general

1 Triggers, procedimientos y funciones en MySQL **5**

1.1 Procedimientos . . . 5
1.2 Funciones . . . 6
1.3 Estructuras de control . . . 6
1.3.1 Instrucciones condicionales . . . 6
1.3.1.1 `IF-THEN-ELSE` . . . 6
1.3.1.2 `CASE` . . . 7
1.3.2 Instrucciones repetitivas o bucles . . . 7
1.3.2.1 `LOOP` . . . 7
1.3.2.2 `REPEAT` . . . 7
1.3.2.3 `WHILE` . . . 8
1.4 Manejo de errores . . . 8
1.4.1 `DECLARE ... HANDLER` . . . 8
1.4.2 Ejemplo 1 - `DECLARE CONTINUE HANDLER` . . . 10
1.4.3 Ejemplo 2 - `DECLARE EXIT HANDLER` . . . 10
1.5 Cómo realizar transacciones con procedimientos almacenados . . . 11
1.6 Cursores . . . 12
1.6.1 Operaciones con cursores . . . 12
1.6.1.1 `DECLARE` . . . 12
1.6.1.2 `OPEN` . . . 13
1.6.1.3 `FETCH` . . . 13
1.6.1.4 `CLOSE` . . . 13
1.7 *Triggers* . . . 15
1.8 Ejercicios . . . 16
1.8.1 Procedimientos sin sentencias SQL . . . 16
1.8.2 Procedimientos con sentencias SQL . . . 19
1.8.3 Funciones sin sentencias SQL . . . 29
1.8.4 Funciones con sentencias SQL . . . 32
1.8.5 Manejo de errores en MySQL . . . 36
1.8.6 Transacciones con procedimientos almacenados . . . 37
1.8.7 Cursores . . . 40
1.8.8 *Triggers* . . . 45
1.9 Ejercicios de repaso . . . 51
1.10 Recursos . . . 54

2 Licencia **55**

Índice de cuadros

Índice de figuras

Capítulo 1

Triggers, procedimientos y funciones en MySQL

1.1 Procedimientos

```
CREATE
    [DEFINER = { user | CURRENT_USER }]
    PROCEDURE sp_name ([proc_parameter[,...]])
    [characteristic ...] routine_body

proc_parameter:
    [ IN | OUT | INOUT ] param_name type

func_parameter:
    param_name type

type:
    Any valid MySQL data type

characteristic:
    COMMENT 'string'
  | LANGUAGE SQL
  | [NOT] DETERMINISTIC
  | { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
  | SQL SECURITY { DEFINER | INVOKER }

routine_body:
    Valid SQL routine statement
```

Puede encontrar más información en la documentación oficial de MySQL.

1.2 Funciones

```
CREATE
    [DEFINER = { user | CURRENT_USER }]
    FUNCTION sp_name ([func_parameter[,...]])
    RETURNS type
    [characteristic ...] routine_body

proc_parameter:
    [ IN | OUT | INOUT ] param_name type

func_parameter:
    param_name type

type:
    Any valid MySQL data type

characteristic:
    COMMENT 'string'
  | LANGUAGE SQL
  | [NOT] DETERMINISTIC
  | { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
  | SQL SECURITY { DEFINER | INVOKER }

routine_body:
    Valid SQL routine statement
```

Puede encontrar más información en la documentación oficial de MySQL.

1.3 Estructuras de control

1.3.1 Instrucciones condicionales

1.3.1.1 IF-THEN-ELSE

```
IF search_condition THEN statement_list
    [ELSEIF search_condition THEN statement_list] ...
    [ELSE statement_list]
END IF
```

Puede encontrar más información en la documentación oficial de MySQL.

1.3.1.2 CASE

```
CASE case_value
    WHEN when_value THEN statement_list
    [WHEN when_value THEN statement_list] ...
    [ELSE statement_list]
END CASE
```

Puede encontrar más información en la documentación oficial de MySQL.

1.3.2 Instrucciones repetitivas o bucles

1.3.2.1 LOOP

```
[begin_label:] LOOP
    statement_list
END LOOP [end_label]
```

Ejemplo:

```
CREATE PROCEDURE doiterate(p1 INT)
BEGIN
  label1: LOOP
    SET p1 = p1 + 1;
    IF p1 < 10 THEN
      ITERATE label1;
    END IF;
    LEAVE label1;
  END LOOP label1;
  SET @x = p1;
END;
```

Puede encontrar más información en la documentación oficial de MySQL.

1.3.2.2 REPEAT

```
[begin_label:] REPEAT
    statement_list
UNTIL search_condition
END REPEAT [end_label]
```

Puede encontrar más información en la documentación oficial de MySQL.

1.3.2.3 WHILE

```
[begin_label:] WHILE search_condition DO
    statement_list
END WHILE [end_label]
```

Puede encontrar más información en la documentación oficial de MySQL.

1.4 Manejo de errores

1.4.1 DECLARE ... HANDLER

```
DECLARE handler_action HANDLER
    FOR condition_value [, condition_value] ...
    statement

handler_action:
    CONTINUE
  | EXIT
  | UNDO

condition_value:
    mysql_error_code
  | SQLSTATE [VALUE] sqlstate_value
  | condition_name
  | SQLWARNING
  | NOT FOUND
  | SQLEXCEPTION
```

Las acciones posibles que podemos seleccionar como *handler_action* son:

- `CONTINUE`: La ejecución del programa continúa.
- `EXIT`: Termina la ejecución del programa.
- `UNDO`: No está soportado en MySQL.

Puede encontrar más información en la documentación oficial de MySQL.

Ejemplo indicando el número de error de MySQL:

En este ejemplo estamos declarando un *handler* que se ejecutará cuando se produzca el error `1051` de MySQL, que ocurre cuando se intenta acceder a una tabla que no existe en la base de datos. En este caso la acción del

handler es `CONTINUE` lo que quiere decir que después de ejecutar las instrucciones especificadas en el cuerpo del *handler* el procedimiento almacenado continuará su ejecución.

```
DECLARE CONTINUE HANDLER FOR 1051
BEGIN
  -- body of handler
END;
```

Ejemplo para `SQLSTATE`:

También podemos indicar el valor de la variable `SQLSTATE`. Por ejemplo, cuando se intenta acceder a una tabla que no existe en la base de datos, el valor de la variable `SQLSTATE` es `42S02`.

```
DECLARE CONTINUE HANDLER FOR SQLSTATE '42S02'
BEGIN
  -- body of handler
END;
```

Ejemplo para `SQLWARNING`:

Es equivalente a indicar todos los valores de `SQLSTATE` que empiezan con `01`.

```
DECLARE CONTINUE HANDLER FOR SQLWARNING
BEGIN
  -- body of handler
END;
```

Ejemplo para `NOT FOUND`:

Es equivalente a indicar todos los valores de `SQLSTATE` que empiezan con `02`. Lo usaremos cuando estemos trabajando con cursores para controlar qué ocurre cuando un cursor alcanza el final del *data set*. Si no hay más filas disponibles en el cursor, entonces ocurre una condición de `NO DATA` con un valor de `SQLSTATE` igual a `02000`. Para detectar esta condición podemos usar un *handler* para controlarlo.

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
BEGIN
  -- body of handler
END;
```

Ejemplo para `SQLEXCEPTION`:

Es equivalente a indicar todos los valores de `SQLSTATE` que empiezan por `00`, `01` y `02`.

```
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
BEGIN
  -- body of handler
END;
```

1.4.2 Ejemplo 1 - DECLARE CONTINUE HANDLER

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
CREATE TABLE test.t (s1 INT, PRIMARY KEY (s1));

-- Paso 3
DELIMITER $$
CREATE PROCEDURE handlerdemo ()
BEGIN
  DECLARE CONTINUE HANDLER FOR SQLSTATE '23000' SET @x = 1;
  SET @x = 1;
  INSERT INTO test.t VALUES (1);
  SET @x = 2;
  INSERT INTO test.t VALUES (1);
  SET @x = 3;
END
$$

DELIMITER ;
CALL handlerdemo();
SELECT @x;
```

¿Qué valor devolvería la sentencia `SELECT @x`?

1.4.3 Ejemplo 2 - DECLARE EXIT HANDLER

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
```

```
CREATE TABLE test.t (s1 INT, PRIMARY KEY (s1));

-- Paso 3
DELIMITER $$
CREATE PROCEDURE handlerdemo ()
BEGIN
  DECLARE EXIT HANDLER FOR SQLSTATE '23000' SET @x = 1;
  SET @x = 1;
  INSERT INTO test.t VALUES (1);
  SET @x = 2;
  INSERT INTO test.t VALUES (1);
  SET @x = 3;
END
$$

DELIMITER ;
CALL handlerdemo();
SELECT @x;
```

¿Qué valor devolvería la sentencia `SELECT @x`?

1.5 Cómo realizar transacciones con procedimientos almacenados

Podemos utilizar el manejo de errores para decidir si hacemos `ROLLBACK` de una transacción. En el siguiente ejemplo vamos a capturar los errores que se produzcan de tipo `SQLEXCEPTION` y `SQLWARNING`.

Ejemplo:

```
DELIMITER $$
CREATE PROCEDURE transaccion_en_mysql()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
      -- ERROR
      ROLLBACK;
  END;

  DECLARE EXIT HANDLER FOR SQLWARNING
  BEGIN
      -- WARNING
  ROLLBACK;
  END;

  START TRANSACTION;
    -- Sentencias SQL
  COMMIT;
```

```
END
$$
```

En lugar de tener un manejador para cada tipo de error, podemos tener uno común para todos los casos.

```
DELIMITER $$
CREATE PROCEDURE transaccion_en_mysql()
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION, SQLWARNING
    BEGIN
      -- ERROR, WARNING
      ROLLBACK;
  END;

  START TRANSACTION;
    -- Sentencias SQL
  COMMIT;
END
$$
```

1.6 Cursores

Los cursores nos permiten almacenar una conjunto de filas de una tabla en una estructura de datos que podemos ir recorriendo de forma secuencial.

Los cursores tienen las siguientes propiedades:

- *Asensitive*: The server may or may not make a copy of its result table.
- *Read only*: son de sólo lectura. No permiten actualizar los datos.
- *Nonscrollable*: sólo pueden ser recorridos en una dirección y no podemos saltarnos filas.

Cuando declaramos un cursor dentro de un procedimiento almacenado debe aparecer antes de las declaraciones de los manejadores de errores (`HANDLER`) y después de la declaración de variables locales.

1.6.1 Operaciones con cursores

Las operaciones que podemos hacer con los cursores son las siguientes:

1.6.1.1 DECLARE

El primer paso que tenemos que hacer para trabajar con cursores es declararlo. La sintaxis para declarar un cursor es:

```
DECLARE cursor_name CURSOR FOR select_statement
```

1.6.1.2 OPEN

Una vez que hemos declarado un cursor tenemos que abrirlo con `OPEN`.

```
OPEN cursor_name
```

1.6.1.3 FETCH

Una vez que el cursor está abierto podemos ir obteniendo cada una de las filas con `FETCH`. La sintaxis es la siguiente:

```
FETCH [[NEXT] FROM] cursor_name INTO var_name [, var_name] ...
```

Cuando se está recorriendo un cursor y no quedan filas por recorrer se lanza el error `NOT FOUND`, que se corresponde con el valor `SQLSTATE` «02000». Por eso cuando estemos trabajando con cursores será necesario declarar un *handler* para manejar este error.

```
DECLARE CONTINUE HANDLER FOR NOT FOUND ...
```

1.6.1.4 CLOSE

Cuando hemos terminado de trabajar con un cursor tenemos que cerrarlo.

```
CLOSE cursor_name
```

Ejemplo:

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
CREATE TABLE t1 (
  id INT UNSIGNED PRIMARY KEY,
```

```
    data VARCHAR(16)
);

CREATE TABLE t2 (
  i INT UNSIGNED
);

CREATE TABLE t3 (
  data VARCHAR(16),
    i INT UNSIGNED
);

INSERT INTO t1 VALUES (1, 'A');
INSERT INTO t1 VALUES (2, 'B');

INSERT INTO t2 VALUES (10);
INSERT INTO t2 VALUES (20);

-- Paso 3
DELIMITER $$
DROP PROCEDURE IF EXISTS curdemo$$
CREATE PROCEDURE curdemo()
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE a CHAR(16);
  DECLARE b, c INT;
  DECLARE cur1 CURSOR FOR SELECT id,data FROM test.t1;
  DECLARE cur2 CURSOR FOR SELECT i FROM test.t2;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

  OPEN cur1;
  OPEN cur2;

  read_loop: LOOP
    FETCH cur1 INTO b, a;
    FETCH cur2 INTO c;
    IF done THEN
      LEAVE read_loop;
    END IF;
    IF b < c THEN
      INSERT INTO test.t3 VALUES (a,b);
    ELSE
      INSERT INTO test.t3 VALUES (a,c);
    END IF;
  END LOOP;

  CLOSE cur1;
  CLOSE cur2;
END
```

```
-- Paso 4
DELIMITER ;
CALL curdemo();

SELECT * FROM t3;
```

Solución utilizando un bucle `WHILE`:

```
DELIMITER $$
DROP PROCEDURE IF EXISTS curdemo$$
CREATE PROCEDURE curdemo()
BEGIN
  DECLARE done INT DEFAULT FALSE;
  DECLARE a CHAR(16);
  DECLARE b, c INT;
  DECLARE cur1 CURSOR FOR SELECT id,data FROM test.t1;
  DECLARE cur2 CURSOR FOR SELECT i FROM test.t2;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;

  OPEN cur1;
  OPEN cur2;

  WHILE done = FALSE DO
    FETCH cur1 INTO b, a;
    FETCH cur2 INTO c;

    IF done = FALSE THEN
    IF b < c THEN
      INSERT INTO test.t3 VALUES (a,b);
    ELSE
      INSERT INTO test.t3 VALUES (a,c);
    END IF;
  END IF;
  END WHILE;

  CLOSE cur1;
  CLOSE cur2;
END;
```

1.7 *Triggers*

```
CREATE
```

```
    [DEFINER = { user | CURRENT_USER }]
    TRIGGER trigger_name
    trigger_time trigger_event
    ON tbl_name FOR EACH ROW
    [trigger_order]
    trigger_body

trigger_time: { BEFORE | AFTER }

trigger_event: { INSERT | UPDATE | DELETE }

trigger_order: { FOLLOWS | PRECEDES } other_trigger_name
```

Un *trigger* es un objeto de la base de datos que está asociado con una tabla y que se activa cuando ocurre un evento sobre la tabla.

Los eventos que pueden ocurrir sobre la tabla son:

- `INSERT`: El *trigger* se activa cuando se inserta una nueva fila sobre la tabla asociada.
- `UPDATE`:
- `DELETE`:

1.8 Ejercicios

1.8.1 Procedimientos sin sentencias SQL

1. Escribe un procedimiento que no tenga ningún parámetro de entrada ni de salida y que muestre el texto `¡Hola mundo!`.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS hola_mundo$$
CREATE PROCEDURE hola_mundo()
BEGIN
  SELECT `¡Hola mundo!';
END
$$
```

Para comprobar el funcionamiento del procedimiento podemos ejecutar:

```
CALL hola_mundo();
```

2. Escribe un procedimiento que reciba un número real de entrada y muestre un mensaje indicando si el número es positivo, negativo o cero.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS comprobar_numero$$
CREATE PROCEDURE comprobar_numero(IN numero FLOAT)
BEGIN
  IF numero > 0 THEN
    SELECT 'Número positivo';
  ELSEIF numero < 0 THEN
    SELECT 'Número negativo';
  ELSE
    SELECT 'El número es 0';
  END IF;
END
$$
```

Para comprobar el funcionamiento del procedimiento podemos ejecutar:

```
CALL comprobar_numero(-4);
```

3. Modifique el procedimiento diseñado en el ejercicio anterior para que tenga un parámetro de entrada, con el valor un número real, y un parámetro de salida, con una cadena de caracteres indicando si el número es positivo, negativo o cero.

```
DROP PROCEDURE  IF exists comprobar_numero;
DELIMITER $$
CREATE PROCEDURE comprobar_numero (IN numero FLOAT, OUT signo VARCHAR(25))
BEGIN
  IF numero < 0 THEN
    SET signo = 'negativo';
  ELSEIF numero > 0 then
        SET signo = 'positivo';
    ELSEIF numero = 0 then
        SET signo = 'cero';
  END IF;
END
$$
```

Para comprobar el funcionamiento del procedimiento podemos ejecutar:

```
CALL comprobar_numero(0, @numero);
SELECT @numero;
```

4. Escribe un procedimiento que reciba un número real de entrada, que representa el valor de la nota de un alumno, y muestre un mensaje indicando qué nota ha obtenido teniendo en cuenta las siguientes condiciones:

- [0,5) = Insuficiente
- [5,6) = Aprobado
- [6, 7) = Bien
- [7, 9) = Notable
- [9, 10] = Sobresaliente
- En cualquier otro caso la nota no será válida.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS calcular_nota$$
CREATE PROCEDURE calcular_nota(IN nota FLOAT)
BEGIN
  DECLARE calificacion VARCHAR(50);

  IF nota >= 0 AND nota < 5 THEN
    SET calificacion = 'Suspenso';
  ELSEIF nota >=5 AND nota < 6 THEN
    SET calificacion = 'Aprobado';
  ELSEIF nota >=6 AND nota < 7 THEN
    SET calificacion = 'Bien';
  ELSEIF nota >= 7 AND nota < 9 THEN
    SET calificacion = 'Notable';
  ELSEIF nota >=9 AND nota <= 10 THEN
    SET calificacion = 'Sobresaliente';
  ELSE
    SET calificacion = 'Nota no válida';
  END IF;

    SELECT calificacion;
END
$$
```

Para comprobar el funcionamiento del procedimiento podemos ejecutar:

```
CALL calcular_nota(3);
```

5. Modifique el procedimiento diseñado en el ejercicio anterior para que tenga un parámetro de entrada, con el valor de la nota en formato numérico y un parámetro de salida, con una cadena de texto indicando la nota correspondiente.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS calcular_nota$$
CREATE PROCEDURE calcular_nota(IN nota FLOAT, OUT nota_texto VARCHAR(25))
BEGIN
  IF nota >= 0 AND nota < 5 THEN
```

```
    SET nota_texto = 'Insuficiente';
  ELSEIF nota >= 5 AND nota < 6 THEN
    SET nota_texto = 'Aprobado';
  ELSEIF nota >= 6 AND nota < 7 THEN
    SET nota_texto = 'Bien';
  ELSEIF nota >= 7 AND nota < 9 THEN
    SET nota_texto = 'Notable';
  ELSEIF nota >= 9 AND nota <= 10 THEN
    SET nota_texto = 'Sobresaliente';
  ELSE
    SET nota_texto = 'Nota no válida';
  END IF;
END
$$

Para comprobar el funcionamiento del procedimiento podemos ejecutar:

CALL calcular_nota(9.5, @nota_de_pepe);
SELECT @nota_de_pepe;
```

1.8.2 Procedimientos con sentencias SQL

1. Escribe un procedimiento que reciba el nombre de un país como parámetro de entrada y realice una consulta sobre la tabla `cliente` para obtener todos los clientes que existen en la tabla de ese país.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS clientes_del_pais$$
CREATE PROCEDURE clientes_del_pais(IN pais VARCHAR(120))
BEGIN
  SELECT * FROM cliente WHERE cliente.pais = pais;
END
$$
```

Tenga en cuenta que en este caso el nombre de la columna (`pais`) coincide con el nombre del parámetro (`pais`). Para poder distinguirlos correctamente usamos el nombre de la tabla cuando queramos hacer referencia a la columna. Por ejemplo, en este caso tendremos `cliente.pais` para hacer referencia a la columna y `pais` para hacer referencia al parámetro de entrada.

Para comprobar el funcionamiento del procedimiento podemos ejecutar:

```
CALL clientes_del_pais('Spain');
```

2. Escribe un procedimiento que reciba como parámetro de entrada una forma de pago, que será una cadena de caracteres (Ejemplo: `PayPal`, `Transferencia`, etc). Y devuelva como salida el pago de máximo valor realizado para esa forma de pago. Deberá hacer uso de la tabla `pago` de la base de datos `jardineria`.

```
DELIMITER $$
DROP PROCEDURE  IF exists calcular_maximo_pago$$
CREATE PROCEDURE calcular_maximo_pago(IN forma_pago VARCHAR(50), OUT maximo FLOAT)
BEGIN
    SET maximo = (
    SELECT MAX(total)
    FROM pago
    WHERE pago.forma_pago = forma_pago);
END
$$

DELIMITER ;
CALL calcular_maximo_pago('PayPal', @maximo);
SELECT @maximo;
```

3. Escribe un procedimiento que reciba como parámetro de entrada una forma de pago, que será una cadena de caracteres (Ejemplo: `PayPal`, `Transferencia`, etc). Y devuelva como salida los siguientes valores teniendo en cuenta la forma de pago seleccionada como parámetro de entrada:

- el pago de máximo valor,
- el pago de mínimo valor,
- el valor medio de los pagos realizados,
- la suma de todos los pagos,
- el número de pagos realizados para esa forma de pago.

Deberá hacer uso de la tabla `pago` de la base de datos `jardineria`.

```
/* Solución con SET */
USE jardineria;

DROP PROCEDURE IF EXISTS calcular_estadísticas;
DELIMITER $$
CREATE PROCEDURE calcular_estadísticas(IN forma_pago VARCHAR(50),
                    OUT máximo FLOAT,
                    OUT mínimo FLOAT,
                    OUT media FLOAT,
                                    OUT suma FLOAT,
                                    OUT numero_pagos INTEGER)
BEGIN
  SET máximo = (
    SELECT MAX(total)
    FROM pago
    WHERE pago.forma_pago = forma_pago);

  SET mínimo = (
    SELECT MIN(total)
```

```
    FROM pago
    WHERE pago.forma_pago = forma_pago);

  SET media = (
    SELECT AVG(total)
    FROM pago
    WHERE pago.forma_pago = forma_pago);

  SET suma = (
    SELECT SUM(total)
    FROM pago
    WHERE pago.forma_pago = forma_pago);

  SET numero_pagos = (
    SELECT COUNT(total)
    FROM pago
    WHERE pago.forma_pago = forma_pago);
END
$$

CALL calcular_estadísticas('Paypal', @máximo, @mínimo, @media, @suma,
    @numero_pagos);
SELECT @máximo, @mínimo, @media, @suma, @numero_pagos;

/* Solución con INTO */
USE jardineria;

DROP PROCEDURE IF EXISTS calcular_estadísticas;
DELIMITER $$
CREATE PROCEDURE calcular_estadísticas(IN forma_pago VARCHAR(50),
                    OUT máximo FLOAT,
                    OUT mínimo FLOAT,
                    OUT media FLOAT,
                                    OUT suma FLOAT,
                                    OUT numero_pagos INTEGER)
BEGIN
  SELECT MAX(total), MIN(total), AVG(total), SUM(total), COUNT(total)
  INTO máximo, mínimo, media, suma, numero_pagos
    FROM pago
  WHERE pago.forma_pago = forma_pago;
END
$$

CALL calcular_estadísticas('Paypal', @máximo, @mínimo, @media, @suma,
    @numero_pagos);
SELECT @máximo, @mínimo, @media, @suma, @numero_pagos;
```

4. Crea una **base de datos** llamada `procedimientos` que contenga una **tabla** llamada `cuadrados`. La tabla `cuadrados` debe tener dos columnas de tipo `INT UNSIGNED`, una columna llamada `número` y otra

columna llamada `cuadrado`.

Una vez creada la base de datos y la tabla deberá **crear un procedimiento** llamado `calcular_cuadrados` con las siguientes características. El procedimiento recibe un parámetro de entrada llamado `tope` de tipo `INT UNSIGNED` y calculará el valor de los cuadrados de los primeros números naturales hasta el valor introducido como parámetro. El valor del números y de sus cuadrados deberán ser almacenados en la tabla `cuadrados` que hemos creado previamente.

Tenga en cuenta que el procedimiento deberá eliminar el contenido actual de la tabla antes de insertar los nuevos valores de los cuadrados que va a calcular.

Utilice un bucle `WHILE` para resolver el procedimiento.

```
-- Paso 1
DROP DATABASE IF EXISTS procedimientos;
CREATE DATABASE procedimientos;
USE procedimientos;

-- Paso 2
CREATE TABLE cuadrados (
  numero INT UNSIGNED,
    cuadrado INT UNSIGNED
);

-- Paso 3
DELIMITER $$
DROP PROCEDURE IF EXISTS calcular_cuadrados$$
CREATE PROCEDURE calcular_cuadrados(IN tope INT UNSIGNED)
BEGIN
  DECLARE numero INT UNSIGNED;

  DELETE FROM cuadrados;

    SET numero = 1;
    WHILE numero <= tope DO
    INSERT INTO cuadrados VALUES (numero, numero*numero);
    SET numero = numero + 1;
    END WHILE;
END

-- Paso 4
DELIMITER ;
CALL calcular_cuadrados(10);
SELECT * FROM cuadrados;
```

Nota:

Para poder ejecutar la sentencia:

```
DELETE FROM cuadrados;
```

en MySQL Workbench debemos tener deshabilitada la opción de ejecución en modo seguro, que es la opción por defecto. Para deshabilitar el modo seguro debemos hacerlo desde: *Edit -> Preferences -> SQL Editor -> Other*

5. Utilice un bucle `REPEAT` para resolver el procedimiento del ejercicio anterior.

```
-- Paso 1
DROP DATABASE IF EXISTS procedimientos;
CREATE DATABASE procedimientos;
USE procedimientos;

-- Paso 2
CREATE TABLE cuadrados (
  numero INT UNSIGNED,
    cuadrado INT UNSIGNED
);

-- Paso 3
DELIMITER $$
DROP PROCEDURE IF EXISTS calcular_cuadrados$$
CREATE PROCEDURE calcular_cuadrados(IN tope INT UNSIGNED)
BEGIN
  DECLARE numero INT UNSIGNED;

  DELETE FROM cuadrados;

    SET numero = 1;
    REPEAT
    INSERT INTO cuadrados VALUES (numero, numero*numero);
    SET numero = numero + 1;
  UNTIL numero > tope
    END REPEAT;
END
```

6. Utilice un bucle `LOOP` para resolver el procedimiento del ejercicio anterior.

```
-- Paso 1
DROP DATABASE IF EXISTS procedimientos;
CREATE DATABASE procedimientos;
USE procedimientos;

-- Paso 2
CREATE TABLE cuadrados (
  numero INT UNSIGNED,
    cuadrado INT UNSIGNED
```

```
);

-- Paso 3
DELIMITER $$
DROP PROCEDURE IF EXISTS calcular_cuadrados$$
CREATE PROCEDURE calcular_cuadrados(IN tope INT UNSIGNED)
BEGIN
  DECLARE numero INT UNSIGNED;

  DELETE FROM cuadrados;

    SET numero = 1;
    bucleloop: LOOP
    IF numero <= tope DO
          INSERT INTO cuadrados VALUES (numero, numero*numero);
      SET numero = numero + 1;
      ITERATE bucleloop;
        END IF;
    LEAVE bucleloop;
    END LOOP;
END

-- Paso 4
DELIMITER ;
CALL calcular_cuadrados(10);
SELECT * FROM cuadrados;
```

7. Crea una **base de datos** llamada `procedimientos` que contenga una **tabla** llamada `pares` y otra **tabla** llamada `impares`. Las dos tablas deben tener única columna llamada `número` y el tipo de dato de esta columna debe ser `INT UNSIGNED`.

Una vez creada la base de datos y las tablas deberá **crear un procedimiento** llamado `calcular_pares_impares` con las siguientes características. El procedimiento recibe un parámetro de entrada llamado `tope` de tipo `INT UNSIGNED` y deberá almacenar en la tabla `pares` aquellos números pares que existan entre el número 1 el valor introducido como parámetro. Habrá que realizar la misma operación para almacenar los números impares en la tabla `impares`.

Tenga en cuenta que el procedimiento deberá eliminar el contenido actual de las tablas antes de insertar los nuevos valores.

Utilice un bucle `WHILE` para resolver el procedimiento.

```
-- Paso 1
DROP DATABASE IF EXISTS procedimientos;
CREATE DATABASE procedimientos;
USE procedimientos;

-- Paso 2
```

```
CREATE TABLE pares (
  numero INT UNSIGNED
);

CREATE TABLE impares (
  numero INT UNSIGNED
);

-- Paso 3
DELIMITER $$
DROP PROCEDURE IF EXISTS calcular_pares_impares$$
CREATE PROCEDURE calcular_pares_impares(IN tope INT UNSIGNED)
BEGIN
  DECLARE numero INT UNSIGNED;

  DELETE FROM pares;
    DELETE FROM impares;

    SET numero = 0;
    WHILE numero <= tope DO

        IF numero % 2 = 0 THEN
      -- Si el número es par
            INSERT INTO pares VALUES (numero);
    ELSE
      -- Si el número es impar
            INSERT INTO impares VALUES (numero);
    END IF;

    SET numero = numero + 1;
    END WHILE;
END
```

Nota:

Para poder ejecutar las sentencias:

```
DELETE FROM pares;
DELETE FROM impares;
```

en MySQL Workbench debemos tener deshabilitada la opción de ejecución en modo seguro, que es la opción por defecto. Para deshabilitar el modo seguro debemos hacerlo desde: *Edit -> Preferences -> SQL Editor -> Other*

Para comprobar que el procedimiento funciona correctamente podemos comprobarlo haciendo:

```
CALL calcular_pares_impares(7);
SELECT * FROM pares;
```

```
SELECT * FROM impares;
```

8. Utilice un bucle `REPEAT` para resolver el procedimiento del ejercicio anterior.

```
-- Paso 1
DROP DATABASE IF EXISTS procedimientos;
CREATE DATABASE procedimientos;
USE procedimientos;

-- Paso 2
CREATE TABLE pares (
  numero INT UNSIGNED
);

CREATE TABLE impares (
  numero INT UNSIGNED
);

-- Paso 3
DELIMITER $$
DROP PROCEDURE IF EXISTS calcular_pares_impares$$
CREATE PROCEDURE calcular_pares_impares(IN tope INT UNSIGNED)
BEGIN
  DECLARE numero INT UNSIGNED;

  DELETE FROM pares;
    DELETE FROM impares;

    SET numero = 0;
    REPEAT
        IF numero % 2 = 0 THEN
            INSERT INTO pares VALUES (numero);
    ELSE
            INSERT INTO impares VALUES (numero);
    END IF;

    SET numero = numero + 1;
    UNTIL numero > tope
    END REPEAT;
END
```

Nota:

Para poder ejecutar las sentencias:

```
DELETE FROM pares;
DELETE FROM impares;
```

en MySQL Workbench debemos tener deshabilitada la opción de ejecución en modo seguro, que es la opción por defecto. Para deshabilitar el modo seguro debemos hacerlo desde: *Edit -> Preferences -> SQL Editor -> Other*

Para comprobar que el procedimiento funciona correctamente podemos comprobarlo haciendo:

```
CALL calcular_pares_impares(7);
SELECT * FROM pares;
SELECT * FROM impares;
```

9. Utilice un bucle `LOOP` para resolver el procedimiento del ejercicio anterior.
10. Crea una **base de datos** llamada `procedimientos` que contenga una **tabla** llamada `ejercicio`. La tabla debe tener una única columna llamada `número` y el tipo de dato de esta columna debe ser `INT UNSIGNED` .

Una vez creada la base de datos y la tabla deberá **crear un procedimiento** llamado `calcular_números` con las siguientes características. El procedimiento recibe un parámetro de entrada llamado `valor_inicial` de tipo `INT UNSIGNED` y deberá almacenar en la tabla `ejercicio` toda la secuencia de números desde el valor inicial pasado como entrada hasta el 1.

Tenga en cuenta que el procedimiento deberá eliminar el contenido actual de las tablas antes de insertar los nuevos valores.

Utilice un bucle `WHILE` para resolver el procedimiento.

11. Utilice un bucle `REPEAT` para resolver el procedimiento del ejercicio anterior.

```
-- Paso 1
DROP DATABASE IF EXISTS procedimientos;
CREATE DATABASE procedimientos;
USE procedimientos;

-- Paso 2
CREATE TABLE ejercicio (
  numero INT UNSIGNED
);

-- Paso 3
DELIMITER $$
DROP PROCEDURE IF EXISTS calcular_numeros$$
CREATE PROCEDURE calcular_numeros(IN valor_inicial INT UNSIGNED)
BEGIN
  DECLARE numero INT UNSIGNED;

  DELETE FROM ejercicio;
```

```
    SET numero = valor_inicial;

    REPEAT
    INSERT INTO ejercicio VALUES (numero);
        SET numero = numero - 1;
    UNTIL numero < 1
    END REPEAT;
END
```

Para comprobar que el procedimiento funciona correctamente podemos comprobarlo haciendo:

```
CALL calcular_numeros(50);
SELECT * FROM ejercicio;
```

12. Utilice un bucle `LOOP` para resolver el procedimiento del ejercicio anterior.

```
-- Paso 1
DROP DATABASE IF EXISTS procedimientos;
CREATE DATABASE procedimientos;
USE procedimientos;

-- Paso 2
CREATE TABLE ejercicio (
  numero INT UNSIGNED
);

-- Paso 3
DELIMITER $$
DROP PROCEDURE IF EXISTS calcular_numeros$$
CREATE PROCEDURE calcular_numeros(IN valor_inicial INT UNSIGNED)
BEGIN
  DECLARE numero INT UNSIGNED;

  DELETE FROM ejercicio;

    SET numero = valor_inicial;

    bucleloop: LOOP
    INSERT INTO ejercicio VALUES (numero);
        SET numero = numero - 1;

        IF numero < 1 THEN
      LEAVE bucleloop;
        END IF;
    END LOOP;
END
```

```
$$

DELIMITER ;
CALL calcular_numeros(25);
SELECT * FROM ejercicio;
```

1.8.3 Funciones sin sentencias SQL

1. Escribe una función que reciba un número entero de entrada y devuelva `TRUE` si el número es par o `FALSE` en caso contrario.

```
DELIMITER $$
DROP FUNCTION IF EXISTS es_par$$
CREATE FUNCTION es_par(numero INT)
  RETURNS BOOLEAN
BEGIN
  DECLARE par BOOLEAN;

    IF numero % 2 = 0 THEN
    SET par = TRUE;
  ELSE
    SET par = FALSE;
  END IF;

    RETURN par;
END
$$

SELECT es_par(3);
```

2. Escribe una función que devuelva el valor de la hipotenusa de un triángulo a partir de los valores de sus lados.

```
DELIMITER $$
DROP FUNCTION IF EXISTS calcular_hipotenusa$$
CREATE FUNCTION calcular_hipotenusa(x FLOAT, y FLOAT)
  RETURNS FLOAT
BEGIN
  DECLARE hipotenusa FLOAT;
  SET hipotenusa = SQRT(POW(x,2) + POW(y,2));
    RETURN hipotenusa;
END
$$
```

```
SELECT calcular_hipotenusa(10.25, 27.34)
```

3. Escribe una función que reciba como parámetro de entrada un valor numérico que represente un día de la semana y que devuelva una cadena de caracteres con el nombre del día de la semana correspondiente. Por ejemplo, para el valor de entrada `1` debería devolver la cadena `lunes`.

```
DELIMITER $$
DROP FUNCTION IF EXISTS calcular_dia$$
CREATE FUNCTION calcular_dia(dia INT)
  RETURNS VARCHAR(50)
BEGIN
  DECLARE nombre_dia VARCHAR(50);

  CASE dia
    WHEN 1 THEN
      SET nombre_dia = 'Lunes';
    WHEN 2 THEN
      SET nombre_dia = 'Martes';
    WHEN 3 THEN
      SET nombre_dia = 'Miércoles';
    WHEN 4 THEN
      SET nombre_dia = 'Jueves';
    WHEN 5 THEN
      SET nombre_dia = 'Viernes';
    WHEN 6 THEN
      SET nombre_dia = 'Sábado';
    WHEN 7 THEN
      SET nombre_dia = 'Domingo';
    ELSE
      SET nombre_dia = 'Valor no válido';
    END CASE;

    RETURN nombre_dia;
END
$$

DELIMITER ;
SELECT calcular_dia(4);
```

4. Escribe una función que reciba tres números reales como parámetros de entrada y devuelva el mayor de los tres.

```
DELIMITER $$
DROP FUNCTION IF EXISTS calcular_mayor$$
CREATE FUNCTION calcular_mayor(a FLOAT, b FLOAT, c FLOAT)
  RETURNS FLOAT
```

```
BEGIN
  DECLARE mayor FLOAT;

  IF a >= b AND a >= c THEN
    SET mayor = a;
  ELSEIF b >= a AND b >= c THEN
    SET mayor = b;
  ELSE
    SET mayor = c;
  END IF;

    RETURN mayor;
END
$$

DELIMITER ;
SELECT calcular_mayor(4, 8, 8);
```

5. Escribe una función que devuelva el valor del área de un círculo a partir del valor del radio que se recibirá como parámetro de entrada.

```
DELIMITER $$
CREATE FUNCTION calcular_area(radio double)
  RETURNS DOUBLE
BEGIN
  DECLARE area DOUBLE;
  SET area = PI() * POW(radio,2);
  RETURN area;
END
$$

DELIMITER ;
SELECT calcular_area(23.34);
```

6. Escribe una función que devuelva como salida el número de años que han transcurrido entre dos fechas que se reciben como parámetros de entrada. Por ejemplo, si pasamos como parámetros de entrada las fechas `2018-01-01` y `2008-01-01` la función tiene que devolver que han pasado 10 años.

Para realizar esta función puede hacer uso de las siguientes funciones que nos proporciona MySQL:

- `DATEDIFF`
- `TRUNCATE`

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;
```

```
-- Paso 2
DELIMITER $$
DROP FUNCTION IF EXISTS calcular_numero_años$$
CREATE FUNCTION calcular_numero_años(fecha_inicial DATE,
                    fecha_final DATE)
  RETURNS INT
BEGIN
  DECLARE dias INT;
    DECLARE años INT;
  SET dias = DATEDIFF(fecha_final, fecha_inicial);
    SET años = dias / 365;
    RETURN años;
END

-- Paso 3
DELIMITER ;
SELECT calcular_numero_años('2000-01-01', '2018-10-01');
```

1.8.4 Funciones con sentencias SQL

1. Escribe una función para la base de datos `tienda` que devuelva el número total de productos que hay en la tabla `productos`.

```
USE tienda;

DELIMITER $$
DROP FUNCTION IF EXISTS calcular_numero_total_productos$$
CREATE FUNCTION calcular_numero_total_productos()
  RETURNS INT
BEGIN
  DECLARE total INT;
    SET total = (SELECT COUNT(*) FROM producto);
    RETURN total;
END

DELIMITER ;
SELECT calcular_numero_total_productos();
```

2. Escribe una función para la base de datos `tienda` que devuelva el valor medio del precio de los productos de un determinado fabricante que se recibirá como parámetro de entrada. El parámetro de entrada será el nombre del fabricante.

Solución con subconsultas:

```
USE tienda;

DELIMITER $$
DROP FUNCTION IF EXISTS calcular_precio_medio$$
CREATE FUNCTION calcular_precio_medio(nombre_fabricante VARCHAR(50))
  RETURNS FLOAT
BEGIN
  DECLARE media FLOAT;

  SET media = (SELECT AVG(precio)
  FROM producto
  WHERE codigo_fabricante = (
    SELECT codigo
    FROM fabricante
    WHERE fabricante.nombre = nombre_fabricante));

    RETURN media;
END

DELIMITER ;
SELECT calcular_precio_medio('Asus');
```

Solución con `INNER JOIN`:

```
USE tienda;

DELIMITER $$
DROP FUNCTION IF EXISTS calcular_precio_medio$$
CREATE FUNCTION calcular_precio_medio(nombre_fabricante VARCHAR(50))
  RETURNS FLOAT
BEGIN
  DECLARE media FLOAT;

  SET media = (SELECT AVG(producto.precio)
    FROM producto INNER JOIN fabricante
    ON fabricante.codigo = producto.codigo_fabricante
    WHERE fabricante.nombre = nombre_fabricante);

    RETURN media;
END

DELIMITER ;
SELECT calcular_precio_medio('Asus');
```

3. Escribe una función para la base de datos `tienda` que devuelva el valor máximo del precio de los productos de un determinado fabricante que se recibirá como parámetro de entrada. El parámetro de entrada será el nombre del fabricante.

Solución con subconsultas:

```
USE tienda;

DELIMITER $$
DROP FUNCTION IF EXISTS calcular_precio_medio$$
CREATE FUNCTION calcular_precio_medio(nombre_fabricante VARCHAR(50))
  RETURNS FLOAT
BEGIN
  DECLARE media FLOAT;

  SET media = (SELECT MAX(precio)
  FROM producto
  WHERE codigo_fabricante = (
    SELECT codigo
    FROM fabricante
    WHERE fabricante.nombre = nombre_fabricante));

    RETURN media;
END

DELIMITER ;
SELECT calcular_precio_medio('Asus');
```

Solución con `INNER JOIN`:

```
USE tienda;

DELIMITER $$
DROP FUNCTION IF EXISTS calcular_precio_medio$$
CREATE FUNCTION calcular_precio_medio(nombre_fabricante VARCHAR(50))
  RETURNS FLOAT
BEGIN
  DECLARE media FLOAT;

  SET media = (SELECT MAX(producto.precio)
    FROM producto INNER JOIN fabricante
    ON fabricante.codigo = producto.codigo_fabricante
    WHERE fabricante.nombre = nombre_fabricante);

    RETURN media;
END

DELIMITER ;
SELECT calcular_precio_medio('Asus');
```

4. Escribe una función para la base de datos `tienda` que devuelva el valor mínimo del precio de los productos de un determinado fabricante que se recibirá como parámetro de entrada. El parámetro de entrada será el nombre del fabricante.

Solución con subconsultas:

```
USE tienda;

DELIMITER $$
DROP FUNCTION IF EXISTS calcular_precio_medio$$
CREATE FUNCTION calcular_precio_medio(nombre_fabricante VARCHAR(50))
  RETURNS FLOAT
BEGIN
  DECLARE media FLOAT;

  SET media = (SELECT MIN(precio)
  FROM producto
  WHERE codigo_fabricante = (
    SELECT codigo
    FROM fabricante
    WHERE fabricante.nombre = nombre_fabricante));

    RETURN media;
END

DELIMITER ;
SELECT calcular_precio_medio('Asus');
```

Solución con `INNER JOIN`:

```
USE tienda;

DELIMITER $$
DROP FUNCTION IF EXISTS calcular_precio_medio$$
CREATE FUNCTION calcular_precio_medio(nombre_fabricante VARCHAR(50))
  RETURNS FLOAT
BEGIN
  DECLARE media FLOAT;

  SET media = (SELECT MIN(producto.precio)
    FROM producto INNER JOIN fabricante
    ON fabricante.codigo = producto.codigo_fabricante
    WHERE fabricante.nombre = nombre_fabricante);

    RETURN media;
END
```

```
DELIMITER ;
SELECT calcular_precio_medio('Asus');
```

1.8.5 Manejo de errores en MySQL

1. Crea una **base de datos** llamada `test` que contenga una **tabla** llamada `alumno`. La tabla debe tener cuatro columnas:

- `id`: entero sin signo (clave primaria).
- `nombre`: cadena de 50 caracteres.
- `apellido1`: cadena de 50 caracteres.
- `apellido2`: cadena de 50 caracteres.

Una vez creada la base de datos y la tabla deberá **crear un procedimiento** llamado `insertar_alumno` con las siguientes características. El procedimiento recibe cuatro parámetros de **entrada** (`id`, `nombre`, `apellido1`, `apellido2`) y los insertará en la tabla `alumno`. El procedimiento devolverá como **salida** un parámetro llamado `error` que tendrá un valor igual a 0 si la operación se ha podido realizar con éxito y un valor igual a 1 en caso contrario.

Deberá manejar los errores que puedan ocurrir cuando se intenta insertar una fila que contiene una clave primaria repetida.

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
CREATE TABLE alumno (
  id INT UNSIGNED PRIMARY KEY,
    nombre VARCHAR(50),
    apellido1 VARCHAR(50),
    apellido2 VARCHAR(50)
);

-- Paso 3
DELIMITER $$
DROP PROCEDURE IF EXISTS insertar_alumno$$
CREATE PROCEDURE insertar_alumno(IN id INT UNSIGNED,
                IN nombre VARCHAR(50),
                                IN apellido1 VARCHAR(50),
                                IN apellido2 VARCHAR(50),
                                OUT error TINYINT UNSIGNED)
BEGIN
  DECLARE CONTINUE HANDLER FOR 1062
    BEGIN
    SET error = 1;
```

```
    END;
    SET error = 0;
  INSERT INTO alumno VALUES (id, nombre, apellido1, apellido2);
END
$$

CALL insertar_alumno(1, 'Pepe', 'López', 'Martínez', @error);
CALL insertar_alumno(2, 'María', 'Sánchez', 'Martínez', @error);
SELECT @error;
SELECT * FROM alumno;
```

1.8.6 Transacciones con procedimientos almacenados

1. Crea una **base de datos** llamada `cine` que contenga **dos tablas** con las siguientes columnas.

Tabla `cuentas`:

- `id_cuenta`: entero sin signo (clave primaria).
- `saldo`: real sin signo.

Tabla `entradas`:

- `id_butaca`: entero sin signo (clave primaria).
- `nif`: cadena de 9 caracteres.

Una vez creada la base de datos y las tablas deberá **crear un procedimiento** llamado `comprar_entrada` con las siguientes características. El procedimiento recibe 3 parámetros de **entrada** (`nif`, `id_cuenta`, `id_butaca`) y devolverá como **salida** un parámetro llamado `error` que tendrá un valor igual a 0 si la compra de la entrada se ha podido realizar con éxito y un valor igual a 1 en caso contrario.

El procedimiento de compra realiza los siguientes pasos:

- Inicia una transacción.
- Actualiza la columna `saldo` de la tabla `cuentas` cobrando 5 euros a la cuenta con el `id_cuenta` adecuado.
- Inserta una una fila en la tabla `entradas` indicando la butaca (`id_butaca`) que acaba de comprar el usuario (`nif`).
- Comprueba si ha ocurrido algún error en las operaciones anteriores. Si no ocurre ningún error entonces aplica un `COMMIT` a la transacción y si ha ocurrido algún error aplica un `ROLLBACK`.

Deberá manejar los siguientes errores que puedan ocurrir durante el proceso.

- ERROR 1264 (Out of range value)
- ERROR 1062 (Duplicate entry for PRIMARY KEY)

```
-- Paso 1
DROP DATABASE IF EXISTS cine;
CREATE DATABASE cine;
USE cine;
```

```
-- Paso 2
CREATE TABLE cuentas (
    id_cuenta INTEGER UNSIGNED PRIMARY KEY,
    saldo DECIMAL(11,2) UNSIGNED CHECK (saldo >= 0)
);

CREATE TABLE entradas (
    id_butaca INTEGER UNSIGNED PRIMARY KEY,
    nif VARCHAR(9)
);

INSERT INTO cuentas VALUES(1, 20);
INSERT INTO cuentas VALUES(2, 10);

-- Solución 1: Con DECLARE CONTINUE HANDLER ...
DELIMITER $$
DROP PROCEDURE IF EXISTS comprar_entrada$$
CREATE PROCEDURE comprar_entrada(IN nif VARCHAR(9),
                IN id_cuenta INT UNSIGNED,
                  IN id_butaca INT UNSIGNED,
                               OUT error TINYINT UNSIGNED)
BEGIN
  DECLARE CONTINUE HANDLER FOR 1264, 1062
    BEGIN
    SET error = 1;
    END;

  -- Inicio la transacción
  START TRANSACTION;
    SET error = 0;

  -- Cobramos la entrada
  UPDATE cuentas SET saldo = saldo - 5
  WHERE cuentas.id_cuenta = id_cuenta;

  -- Reservo la butaca
    INSERT INTO entradas VALUES (id_butaca, nif);

  -- Compruebo si ha ocurrido algún error
    IF error = 0 THEN
    COMMIT;
  ELSE
    ROLLBACK;
  END IF;
END

DELIMITER ;
CALL comprar_entrada('11111111A', 1, 10, @error);
```

```
CALL comprar_entrada('11111111A', 1, 11, @error);
CALL comprar_entrada('11111111A', 1, 12, @error);
CALL comprar_entrada('11111111A', 1, 13, @error);
CALL comprar_entrada('11111111A', 1, 14, @error);
CALL comprar_entrada('22222222B', 2, 10, @error);
SELECT @error;
SELECT * FROM cuentas;
SELECT * FROM entradas;
```

```
-- Solución 2: Con DECLARE EXIT HANDLER ...

DELIMITER $$
DROP PROCEDURE IF EXISTS comprar_entrada$$
CREATE PROCEDURE comprar_entrada(IN nif VARCHAR(9),
                IN id_cuenta INT UNSIGNED,
                  IN id_butaca INT UNSIGNED,
                               OUT error TINYINT UNSIGNED)
BEGIN
  DECLARE EXIT HANDLER FOR 1264, 1062
    BEGIN
    SET error = 1;
        ROLLBACK;
    END;

  -- Inicio la transacción
  START TRANSACTION;
    SET error = 0;

  -- Cobramos la entrada
  UPDATE cuentas SET saldo = saldo - 5
  WHERE cuentas.id_cuenta = id_cuenta;

  -- Reservamos la butaca
    INSERT INTO entradas VALUES (id_butaca, nif);

  COMMIT;
END
```

2. ¿Qué ocurre cuando intentamos comprar una entrada y le pasamos como parámetro un número de cuenta que no existe en la tabla `cuentas`? ¿Ocurre algún error o podemos comprar la entrada?

En caso de que exista algún error, ¿cómo podríamos resolverlo?.

```
DELIMITER $$
DROP PROCEDURE IF EXISTS comprar_entrada$$
CREATE PROCEDURE comprar_entrada(IN nif VARCHAR(9),
```

```
                IN id_cuenta INTEGER UNSIGNED,
                IN id_butaca INTEGER UNSIGNED,
                              OUT error TINYINT)
BEGIN
  DECLARE saldo_cliente DECIMAL UNSIGNED;
    DECLARE CONTINUE HANDLER FOR 1264, 1062
    BEGIN
    SET error = 1;
    END;

    SET error = 0;
    SET saldo_cliente = (SELECT saldo FROM cuentas WHERE cuentas.id_cuenta =
        id_cuenta);

  IF saldo_cliente >= 5 THEN
        START TRANSACTION;

    UPDATE cuentas SET saldo = saldo - 5
    WHERE cuentas.id_cuenta = id_cuenta;

    INSERT INTO entradas VALUES (id_butaca, nif);

    IF error = 0 THEN
      COMMIT;
    ELSE
      ROLLBACK;
    END IF;
  ELSE
    SET error = 1;
    END IF;
END
$$

DELIMITER ;
CALL comprar_entrada('44444444A',3,11, @error);
SELECT * FROM cuentas;
SELECT * FROM entradas;
SELECT @error;
```

1.8.7 Cursores

1. Escribe las sentencias SQL necesarias para **crear una base de datos** llamada `test`, **una tabla** llamada `alumnos` y **4 sentencias de inserción** para inicializar la tabla. La tabla alumnos está formada por las siguientes columnas:

- `id` (entero sin signo y clave primaria)
- `nombre` (cadena de caracteres)

- `apellido1` (cadena de caracteres)
- `apellido2` (cadena de caracteres
- `fecha_nacimiento` (fecha)

Una vez creada la tabla se decide añadir una nueva columna a la tabla llamada `edad` que será un valor calculado a partir de la columna `fecha_nacimiento`. Escriba la sentencia SQL necesaria para **modificar la tabla y añadir la nueva columna**.

Escriba una **función** llamada `calcular_edad` que reciba una fecha y devuelva el número de años que han pasado desde la fecha actual hasta la fecha pasada como parámetro:

- Función: `calcular_edad`
- Entrada: Fecha
- Salida: Número de años (entero)

Ahora escriba un procedimiento que permita calcular la edad de todos los alumnmos que ya existen en la tabla. Para esto será necesario crear un **procedimiento** llamado `actualizar_columna_edad` que calcule la edad de cada alumno y actualice la tabla. Este procedimiento hará uso de la función `calcular_edad` que hemos creado en el paso anterior.

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
CREATE TABLE alumnos (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido1 VARCHAR(50) NOT NULL,
    apellido2 VARCHAR(50),
    fecha_nacimiento DATE NOT NULL
);

-- Paso 3
INSERT INTO alumnos VALUES (1, 'Pepe', 'López', 'López', '2000-01-01');
INSERT INTO alumnos VALUES (2, 'María', 'Martínez', 'Martínez', '1999-07-13');
INSERT INTO alumnos VALUES (3, 'Juan', 'García', 'García', '1998-06-21');
INSERT INTO alumnos VALUES (4, 'Lucía', 'Sáez', 'Sáez', '1997-02-11');

-- Paso 4
ALTER TABLE alumnos ADD edad INT UNSIGNED;

-- Paso 5
DELIMITER $$
DROP FUNCTION IF EXISTS calcular_edad$$
CREATE FUNCTION calcular_edad(fecha DATE)
  RETURNS INT
BEGIN
```

```
  DECLARE edad INT;
    SET edad = TRUNCATE(DATEDIFF(NOW(), fecha) / 365, 0);
    RETURN edad;
END

-- Paso 6
DELIMITER $$
DROP PROCEDURE IF EXISTS actualizar_columna_edad$$
CREATE PROCEDURE actualizar_columna_edad()
BEGIN
  DECLARE fecha_nacimiento DATE;
  DECLARE edad INT;
    DECLARE id INT;
    DECLARE fin INT DEFAULT 0;

  DECLARE cursor_alumnos CURSOR FOR
    SELECT alumnos.id, alumnos.fecha_nacimiento FROM alumnos;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin = 1;

  OPEN cursor_alumnos;
    bucle: LOOP
    FETCH cursor_alumnos INTO id, fecha_nacimiento;

        IF fin = 1 THEN
      LEAVE bucle;
        END IF;

        SET edad = calcular_edad(fecha_nacimiento);

        UPDATE alumnos SET alumnos.edad = edad
        WHERE alumnos.id = id;

    END LOOP;
    CLOSE cursor_alumnos;
END

-- Ejecutamos el procedimiento y comprobamos el resultado
DELIMITER ;
CALL actualizar_columna_edad();

SELECT * FROM alumnos;
```

2. Modifica la tabla `alumnos` del ejercicio anterior para añadir una nueva columna `email`. Una vez que hemos modificado la tabla necesitamos asignarle una dirección de correo electrónico de forma automática.

Escriba un **procedimiento** llamado **crear_email** que dados los parámetros de entrada: `nombre`, `apellido1`, `apellido2` y `dominio`, cree una dirección de email y la devuelva como salida.

- Procedimiento: `crear_email`

- Entrada:
  - `nombre` (cadena de caracteres)
  - `apellido1` (cadena de caracteres)
  - `apellido2` (cadena de caracteres)
  - `dominio` (cadena de caracteres)
- Salida:
  - `email` (cadena de caracteres)

devuelva una dirección de correo electrónico con el siguiente formato:

- El primer carácter del parámetro `nombre`.
- Los tres primeros caracteres del parámetro `apellido1`.
- Los tres primeros caracteres del parámetro `apellido2`.
- El carácter `@`.
- El dominio pasado como parámetro.

Ahora escriba un procedimiento que permita crear un email para todos los alumnmos que ya existen en la tabla. Para esto será necesario crear un **procedimiento** llamado `actualizar_columna_email` que actualice la columna `email` de la tabla `alumnos`. Este procedimiento hará uso del procedimiento `crear_email` que hemos creado en el paso anterior.

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
CREATE TABLE alumnos (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido1 VARCHAR(50) NOT NULL,
    apellido2 VARCHAR(50),
    fecha_nacimiento DATE NOT NULL
);

-- Paso 3
INSERT INTO alumnos VALUES (1, 'Pepe', 'López', 'López', '2000-01-01');
INSERT INTO alumnos VALUES (2, 'María', 'Martínez', 'Martínez', '1999-07-13');
INSERT INTO alumnos VALUES (3, 'Juan', 'García', 'García', '1998-06-21');
INSERT INTO alumnos VALUES (4, 'Lucía', 'Sáez', 'Sáez', '1997-02-11');

-- Paso 4
ALTER TABLE alumnos ADD email VARCHAR(150);

-- Paso 5
DELIMITER $$
DROP PROCEDURE IF EXISTS crear_email$$
CREATE PROCEDURE crear_email(IN nombre VARCHAR(50),
```

```
                IN apellido1 VARCHAR(50),
                IN apellido2 VARCHAR(50),
                IN dominio VARCHAR(50),
                             OUT email VARCHAR(150))
BEGIN
  SET email = CONCAT(SUBSTR(nombre,1,1),
              SUBSTR(apellido1,1,3),
              SUBSTR(apellido2,1,3),
                       '@', dominio);
  SET email = LOWER(email);
END

-- Paso 6
DELIMITER $$
DROP PROCEDURE IF EXISTS actualizar_columna_email$$
CREATE PROCEDURE actualizar_columna_email(IN dominio VARCHAR(50))
BEGIN
  DECLARE termina INT DEFAULT 0;
  DECLARE id INT UNSIGNED;
    DECLARE nombre, apellido1, apellido2 VARCHAR(50);
    DECLARE email VARCHAR(150);

  DECLARE cursor_alumnos CURSOR FOR
    SELECT alumnos.id, alumnos.nombre, alumnos.apellido1, alumnos.apellido2
    FROM alumnos;

    DECLARE CONTINUE HANDLER FOR NOT FOUND SET termina = 1;

    OPEN cursor_alumnos;
    bucle: LOOP
    FETCH cursor_alumnos INTO id, nombre, apellido1, apellido2;

        IF termina = 1 THEN
      LEAVE bucle;
    END IF;

    CALL crear_email(nombre, apellido1, apellido2, dominio, @email);
        UPDATE alumnos SET alumnos.email = @email WHERE alumnos.id = id;

    END LOOP;
    CLOSE cursor_alumnos;
END

DELIMITER ;
CALL actualizar_columna_email('iescelia.org');
SELECT * FROM alumnos;
```

3. Escribe un **procedimiento** llamado `crear_lista_emails_alumnos` que devuelva la lista de emails de la tabla `alumnos` separados por un punto y coma. Ejemplo: `juan@iescelia.org;maria@iescelia`.

`org;pepe@iescelia.org;lucia@iescelia.org`.

1.8.8 *Triggers*

1. Crea una **base de datos** llamada `test` que contenga **una tabla** llamada `alumnos` con las siguientes columnas.

Tabla `alumnos`:

- `id` (entero sin signo)
- `nombre` (cadena de caracteres)
- `apellido1` (cadena de caracteres)
- `apellido2` (cadena de caracteres)
- `nota` (número real)

Una vez creada la tabla escriba **dos triggers** con las siguientes características:

- Trigger 1: `trigger_check_nota_before_insert`
  - Se ejecuta sobre la tabla `alumnos`.
  - Se ejecuta *antes* de una operación de *inserción*.
  - Si el nuevo valor de la nota que se quiere insertar es negativo, se guarda como 0.
  - Si el nuevo valor de la nota que se quiere insertar es mayor que 10, se guarda como 10.
- Trigger2 : `trigger_check_nota_before_update`
  - Se ejecuta sobre la tabla `alumnos`.
  - Se ejecuta *antes* de una operación de *actualización*.
  - Si el nuevo valor de la nota que se quiere actualizar es negativo, se guarda como 0.
  - Si el nuevo valor de la nota que se quiere actualizar es mayor que 10, se guarda como 10.

Una vez creados los triggers escriba varias sentencias de inserción y actualización sobre la tabla `alumnos` y verifica que los *triggers* se están ejecutando correctamente.

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
CREATE TABLE alumnos (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido1 VARCHAR(50) NOT NULL,
    apellido2 VARCHAR(50),
    nota FLOAT
);

-- Paso 3
DELIMITER $$
DROP TRIGGER IF EXISTS trigger_check_nota_before_insert$$
```

```
CREATE TRIGGER trigger_check_nota_before_insert
BEFORE INSERT
ON alumnos FOR EACH ROW
BEGIN
  IF NEW.nota < 0 THEN
    set NEW.nota = 0;
  ELSEIF NEW.nota > 10 THEN
    set NEW.nota = 10;
  END IF;
END

DELIMITER $$
DROP TRIGGER IF EXISTS trigger_check_nota_before_update$$
CREATE TRIGGER trigger_check_nota_before_update
BEFORE UPDATE
ON alumnos FOR EACH ROW
BEGIN
  IF NEW.nota < 0 THEN
    set NEW.nota = 0;
  ELSEIF NEW.nota > 10 THEN
    set NEW.nota = 10;
  END IF;
END

-- Paso 4
DELIMITER ;
INSERT INTO alumnos VALUES (1, 'Pepe', 'López', 'López', -1);
INSERT INTO alumnos VALUES (2, 'María', 'Sánchez', 'Sánchez', 11);
INSERT INTO alumnos VALUES (3, 'Juan', 'Pérez', 'Pérez', 8.5);

-- Paso 5
SELECT * FROM alumnos;

-- Paso 6
UPDATE alumnos SET nota = -4 WHERE id = 3;
UPDATE alumnos SET nota = 14 WHERE id = 3;
UPDATE alumnos SET nota = 9.5 WHERE id = 3;

-- Paso 7
SELECT * FROM alumnos;
```

2. Crea una **base de datos** llamada `test` que contenga **una tabla** llamada `alumnos` con las siguientes columnas.

Tabla `alumnos`:

- `id` (entero sin signo)
- `nombre` (cadena de caracteres)
- `apellido1` (cadena de caracteres)

- `apellido2` (cadena de caracteres)
- `email` (cadena de caracteres)

Escriba un **procedimiento** llamado **crear_email** que dados los parámetros de entrada: `nombre`, `apellido1`, `apellido2` y `dominio`, cree una dirección de email y la devuelva como salida.

- Procedimiento: `crear_email`
- Entrada:
  - `nombre` (cadena de caracteres)
  - `apellido1` (cadena de caracteres)
  - `apellido2` (cadena de caracteres)
  - `dominio` (cadena de caracteres)
- Salida:
  - `email` (cadena de caracteres)

devuelva una dirección de correo electrónico con el siguiente formato:

- El primer carácter del parámetro `nombre`.
- Los tres primeros caracteres del parámetro `apellido1`.
- Los tres primeros caracteres del parámetro `apellido2`.
- El carácter `@`.
- El dominio pasado como parámetro.

Una vez creada la tabla escriba **un trigger** con las siguientes características:

- Trigger: `trigger_crear_email_before_insert`
  - Se ejecuta sobre la tabla `alumnos`.
  - Se ejecuta *antes* de una operación de *inserción*.
  - Si el nuevo valor del email que se quiere insertar es `NULL`, entonces se le creará automáticamente una dirección de email y se insertará en la tabla.
  - Si el nuevo valor del email no es `NULL` se guardará en la tabla el valor del email.

Nota: Para crear la nueva dirección de email se deberá hacer uso del procedimiento `crear_email`.

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
CREATE TABLE alumnos (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido1 VARCHAR(50) NOT NULL,
    apellido2 VARCHAR(50),
    email VARCHAR(128)
);

-- Paso 3
```

```
DELIMITER $$
DROP PROCEDURE IF EXISTS crear_email$$
CREATE PROCEDURE crear_email(IN nombre VARCHAR(50),
               IN apellido1 VARCHAR(50),
               IN apellido2 VARCHAR(50),
               IN dominio VARCHAR(50),
                           OUT email VARCHAR(150))
BEGIN
  SET email = CONCAT(SUBSTR(nombre,1,1),
             SUBSTR(apellido1,1,3),
             SUBSTR(apellido2,1,3),
                      '@', dominio);
  SET email = LOWER(email);
END

-- Paso 4
DELIMITER $$
DROP TRIGGER IF EXISTS trigger_crear_email_before_insert$$
CREATE TRIGGER trigger_crear_email_before_insert
BEFORE INSERT
ON alumnos FOR EACH ROW
BEGIN
  DECLARE email VARCHAR(150);
  DECLARE dominio VARCHAR(50);
    SET dominio = 'iescelia.org';

  IF NEW.email IS NULL THEN
    CALL crear_email(NEW.nombre,
             NEW.apellido1,
                         NEW.apellido2,
                         dominio,
                         @email);
    SET NEW.email = @email;
    END IF;
END

-- Paso 5
DELIMITER ;
INSERT INTO alumnos VALUES (1,'Pepe', 'López', 'López', NULL);

SELECT * FROM alumnos;
```

3. Modifica el ejercicio anterior y añade un nuevo *trigger* que las siguientes características:

Trigger: `trigger_guardar_email_after_update`:

- Se ejecuta sobre la tabla `alumnos`.
- Se ejecuta *después* de una operación de *actualización*.
- Cada vez que un alumno modifique su dirección de email se deberá insertar un nuevo registro en una tabla llamada `log_cambios_email`.

La tabla `log_cambios_email` contiene los siguientes campos:

- `id`: clave primaria (entero autonumérico)
- `id_alumno`: id del alumno (entero)
- `fecha_hora`: marca de tiempo con el instante del cambio (fecha y hora)
- `old_email`: valor anterior del email (cadena de caracteres)
- `new_email`: nuevo valor con el que se ha actualizado

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
CREATE TABLE alumnos (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido1 VARCHAR(50) NOT NULL,
    apellido2 VARCHAR(50),
    email VARCHAR(128)
);

CREATE TABLE log_cambios_email (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    id_alumno INT UNSIGNED,
    fecha_hora TIMESTAMP,
    old_email VARCHAR(128),
    new_email VARCHAR(128),
    FOREIGN KEY (id_alumno) REFERENCES alumnos(id)
);

-- Paso 3
DELIMITER $$
DROP TRIGGER IF EXISTS trigger_guardar_email_after_update$$
CREATE TRIGGER trigger_guardar_email_after_update
AFTER UPDATE
ON alumnos FOR EACH ROW
BEGIN
  IF OLD.email <> NEW.email THEN
    INSERT INTO log_cambios_email (id_alumno, fecha_hora, old_email, new_email)
    VALUES (OLD.id, NOW(), OLD.email, NEW.email);
  END IF;
END

-- Paso 4
DELIMITER ;
INSERT INTO alumnos VALUES (1, 'Pepe', 'López', 'López', 'pepe@iescelia.org');
```

```
-- Paso 5
UPDATE alumnos SET email = 'pp@iescelia.es' WHERE id = 1;

-- Paso 6
SELECT * FROM alumnos;
SELECT * FROM log_cambios_email;
```

4. Modifica el ejercicio anterior y añade un nuevo *trigger* que las siguientes características:

Trigger: `trigger_guardar_alumnos_eliminados`:

- Se ejecuta sobre la tabla `alumnos`.
- Se ejecuta *después* de una operación de *borrado*.
- Cada vez que se elimine un alumno de la tabla `alumnos` se deberá insertar un nuevo registro en una tabla llamada `log_alumnos_eliminados`.

La tabla `log_alumnos_eliminados` contiene los siguientes campos:

- `id`: clave primaria (entero autonumérico)
- `id_alumno`: id del alumno (entero)
- `fecha_hora`: marca de tiempo con el instante del cambio (fecha y hora)
- `nombre`: nombre del alumno eliminado (cadena de caracteres)
- `apellido1`: primer apellido del alumno eliminado (cadena de caracteres)
- `apellido2`: segundo apellido del alumno eliminado (cadena de caracteres)
- `email`: email del alumno eliminado (cadena de caracteres)

```
-- Paso 1
DROP DATABASE IF EXISTS test;
CREATE DATABASE test;
USE test;

-- Paso 2
CREATE TABLE alumnos (
    id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido1 VARCHAR(50) NOT NULL,
    apellido2 VARCHAR(50),
    email VARCHAR(128)
);

CREATE TABLE log_alumnos_eliminados (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    id_alumno INT UNSIGNED,
    fecha_hora TIMESTAMP,
    nombre VARCHAR(50) NOT NULL,
    apellido1 VARCHAR(50) NOT NULL,
    apellido2 VARCHAR(50),
  email VARCHAR(128)
);
```

```
-- Paso 3
DELIMITER $$
DROP TRIGGER IF EXISTS trigger_guardar_alumnos_eliminados$$
CREATE TRIGGER trigger_guardar_alumnos_eliminados
AFTER DELETE
ON alumnos FOR EACH ROW
BEGIN
  INSERT INTO log_alumnos_eliminados
  VALUES (NULL, OLD.id, NOW(), OLD.nombre, OLD.apellido1, OLD.apellido2, OLD.email
      );
END

-- Paso 4
DELIMITER ;
INSERT INTO alumnos VALUES (1, 'Pepe', 'López', 'López', 'pepe@iescelia.org');

-- Paso 5
DELETE FROM alumnos WHERE id = 1;

-- Paso 6
SELECT * FROM alumnos;
SELECT * FROM log_alumnos_eliminados;
```

1.9 Ejercicios de repaso

1. ¿Qué beneficios nos puede aportar utilizar procedimientos y funciones almacenadas?
2. Según la siguiente sentencia, ¿estamos haciendo una llamada a un procedimiento o a una función?

```
CALL resolver_ejercicio2()
```

3. ¿Cuáles de los siguientes bloques son correctos?

```
1.
LOOP bucle:
  statements
END bucle:

2.
bucle: LOOP
  statements
END bucle;
```

```
3.
bucle:
LOOP bucle;
  statements;
END bucle;
```

4. ¿Pueden aparecer las siguientes sentencias en el mismo bloque de código?

```
DECLARE a INT;
DECLARE a INT;
```

5. ¿Pueden aparecer las siguientes sentencias en el mismo bloque de código?

```
DECLARE a INT;
DECLARE a FLOAT;
```

6. ¿Pueden aparecer las siguientes sentencias en el mismo bloque de código?

```
DECLARE b VARCHAR(20);
DECLARE b HANDLER FOR SQLSTATE '02000';
```

7. ¿Para qué podemos utilizar un cursor en MySQL?
8. ¿Puedo actualizar los datos de un cursor en MySQL? Si fuese posible actualizar los datos de un cursor, ¿se actualizarían automáticamente los datos de la tabla?
9. Cuál o cuáles de los siguientes bucles no está soportado en MySQL: `FOR`, `LOOP`, `REPEAT` y `WHILE`.
10. Si el cuerpo del bucle se debe ejecutar al menos una vez, ¿qué bucle sería más apropiado?
11. ¿Qué valor devuelve la sentencia `SELECT value`?

- 0
- 9
- 10
- `NULL`
- El código entra en un bucle infinito y nunca alcanza la sentencia `SELECT value`

```
DELIMITER $$
CREATE PROCEDURE incrementor (OUT i INT)
BEGIN
  REPEAT
    SET i = i + 1;
  UNTIL i > 9
  END REPEAT;
```

```
END;

DELIMITER $$
CREATE PROCEDURE test ()
BEGIN
  DECLARE value INT default 0;
  CALL incrementor(value);

  -- ¿Qué valor se muestra en esta sentencia?
  SELECT value;
END;

DELIMITER ;
CALL test();
```

11. ¿Qué valor devuelve la sentencia `SELECT value`?

- 0
- 9
- 10
- `NULL`
- El código entra en un bucle infinito y nunca alcanza la sentencia `SELECT value`

```
DELIMITER $$
CREATE PROCEDURE incrementor (IN i INT)
BEGIN
  REPEAT
    SET i = i + 1;
  UNTIL i > 9
  END REPEAT;
END;

DELIMITER $$
CREATE PROCEDURE test ()
BEGIN
  DECLARE value INT default 0;
  CALL incrementor(value);

  -- ¿Qué valor se muestra en esta sentencia?
  SELECT value;
END;

DELIMITER ;
CALL test();
```

12. Realice los siguientes procedimientos y funciones sobre la base de datos `jardineria`.

a)

- Función: `calcular_precio_total_pedido`
- Descripción: Dado un código de pedido la función debe calcular la suma total del pedido. Tenga en cuenta que un pedido puede contener varios productos diferentes y varias cantidades de cada producto.
- Parámetros de entrada: `codigo_pedido` (`INT`)
- Parámetros de salida: El precio total del pedido (`FLOAT`)

b)

- Función: `calcular_suma_pedidos_cliente`
- Descripción: Dado un código de cliente la función debe calcular la suma total de todos los pedidos realizados por el cliente. Deberá hacer uso de la función `calcular_precio_total_pedido` que ha desarrollado en el apartado anterior.
- Parámetros de entrada: `codigo_cliente` (`INT`)
- Parámetros de salida: La suma total de todos los pedidos del cliente (`FLOAT`)

c)

- Función: `calcular_suma_pagos_cliente`
- Descripción: Dado un código de cliente la función debe calcular la suma total de los pagos realizados por ese cliente.
- Parámetros de entrada: `codigo_cliente` (`INT`)
- Parámetros de salida: La suma total de todos los pagos del cliente (`FLOAT`)

d)

- Procedimiento: `calcular_pagos_pendientes`
- Descripción: Deberá calcular los pagos pendientes de todos los clientes. Para saber si un cliente tiene algún pago pendiente deberemos calcular cuál es la cantidad de todos los pedidos y los pagos que ha realizado. Si la cantidad de los pedidos es mayor que la de los pagos entonces ese cliente tiene pagos pendientes.

Deberá insertar en una tabla llamada `clientes_con_pagos_pendientes` los siguientes datos:

- `id_cliente`
- `suma_total_pedidos`
- `suma_total_pagos`
- `pendiente_de_pago`

1.10 Recursos

- MySQL Stored Procedures. Peter Gulutzan.

Capítulo 2

Licencia

Esta obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional.